Date: 25/02/26

Program 15: **User Defined Exception**

Aim: Write a user defined exception class to authenticate the user name and password.

Algorithm:

1. Define two custom exceptions: **nameexception** and **passexception**, both extending **RuntimeException**, and displaying the message passed with the exception.
2. Define a class user with instance variables **name** and **password.**
3. Declare a constructor to initialize the variables.
4. Declare a method **login** within **user** class to check if the provided username and password match the stored values. If they match, print **Login Successful**, otherwise throw a exception indicating invalid username or password.
5. In the **main class:**
    - Prompt the user to enter a name and validate it character by character.
        - If the character is not an alphabet, throw a **nameexception** indicating invalid name.
    - Prompt the user to enter a password and validate it.
        - If the password length is less than 8 characters, throw a **passexception**, indicating minimum length of password.
        - If the password does not contain at least one digit, throw a **passexception**, indicating minimum requirement of 1 digit in the password.
6. Create a **user** object with the provided name and password.
7. Prompt for login username and password.
8. Call the **login** method of the **user** object.

Source Code:

```
import java.util.*;
class nameexception extends RuntimeException
{
    nameexception(String s)
    {
        super(s);
    }
}
class passexception extends RuntimeException
{
    passexception(String s)
    {
        super(s);
    }
}
```

```
class user
{
    String name,password;
    user(String n, String p)
    {
        name=n;
        password=p;
    }
    void login(String n, String p)
    {
        try
        {
            if(name.equals(n)&&password.equals(p))
                System.out.println("\t\tLogin successful");
            else
                throw new passexception("Invalid username or password!!
                                        ");
        }
        catch (passexception e)
        {
            System.out.println("\n"+e.getMessage());
            System.exit(0);
        }
    }
}


class validation
{
    public static void main(String[] args)
    {
        Scanner sc=new Scanner(System.in);
        System.out.print("Enter name: ");
        String s=sc.next();
        try
        {
            for(int i=0;i<s.length();i++)
            {
                char ch=s.charAt(i);
                if((ch>=65&&ch<=90)||(ch>=97&&ch<=122))
                    continue;
                else
```

```
                throw new nameexception("Invalid Name");
        }
    }
    catch (nameexception e)
    {
        System.out.println("\n"+e.getMessage());
        System.exit(0);
    }
    System.out.print("Enter password: ");
    String pass=sc.next();
    try
    {
        int p=0;
        if(pass.length()<8)
            throw new nameexception("Password must have 8 characters
                                    ");
        for(int i=0;i<pass.length();i++)
        {
            char ch=pass.charAt(i);
            if((ch>=48&&ch<=57))
            {
                p=1;
            }
        }
        if(p==0)
            throw new passexception("Password must contain atleast 1
                                     number");
    }
    catch (nameexception e)
    {
        System.out.println("\n"+e.getMessage());
        System.exit(0);
    }
    catch (passexception e)
    {
        System.out.println("\n"+e.getMessage());
        System.exit(0);
    }
    user u1=new user(s,pass);
    System.out.println("\t\tLogin!!");
    System.out.print("Username: ");
    String n1=sc.next();
```

```
            System.out.print("Password: ");
            String p1=sc.next();
            u1.login(n1,p1);
        }
    }
```

Output:

```
Enter name: Dhoni
Enter password: mahi0007
                Login!!
Username: Dhoni
Password: mahi007

Invalid username or password!!
```

```
Enter name: Dhoni
Enter password: mahi

Password must have 8 charachters
```

Result: The program has executed successfully and required output is obtained.

Date: 25/02/26

Program 16: **User Defined Exception** (Negative Numbers)

Aim: Find the average of N positive integers, raising a user defined exception for each negative input.

Algorithm:

1. Define a custom exception class **negative** which extends **RuntimeException**. This exception is thrown when a negative number is encountered.
2. Define the main class **Average**.
3. Prompt the user to enter the number of elements **n**.
4. Create an integer array **nos** of size **n**.
5. Initialize a variable **sum** to store the sum of the entered numbers.
6. Prompt the user to enter **n** numbers and store them in the array **nos**.
   - While taking input, if any number entered is negative, throw a **negative** exception indicating that a negative number is not allowed.
   - If the entered number is positive, add it to the **sum**.
7. Calculate and display the average by dividing the **sum** by the number of elements (**n**).

Source Code:

```
import java.util.*;
class negative extends RuntimeException
{
    negative(String s)
    {
        super(s);
    }
}

class Average
{
    public static void main(String[] args)
    {
        Scanner sc=new Scanner(System.in);
        System.out.print("Enter no:of elements: ");
        int n=sc.nextInt();
        int nos[]=new int[n];
        double sum=0;
        System.out.print("Enter "+n+" numbers: ");
        for(int i=0;i<n;i++)
        {
            nos[i]=sc.nextInt();
            try
```

```
            {
                if(nos[i]<0)
                    throw new negative("Negative number is not allowed
                                        !!!");
                else
                sum+=nos[i];
            }
            catch(negative e)
            {
                System.out.println("\n"+e.getMessage());
                System.exit(0);
            }
        }
        System.out.println("Average: " + String.format("%.2f", sum/n));

    }
}
```

Output:

```
Enter no:of elements: 3
Enter 3 numbers: 4 2 1
Average: 2.33
```

```
Enter no:of elements: 3
Enter 3 numbers: 4 -2 1

Negative number is not allowed!!!
```

Result: The program has executed successfully and required output is obtained.

Date: 04/03/26

Program 17: **Thread Class**

Aim: Define 2 classes; one for generating multiplication table of 5 and other for displaying first N prime numbers. Implement using thread class.

Algorithm:

1. Define a class named **multiple5** that **extends** the **Thread** class.
2. Override the **run**() method in the multiple5 class.
   - Iterate from 1 to 10.
   - For each iteration, print the value of **i multiplied by 5.**
3. Define another class named **Prime** that **extends** the **Thread** class.
4. Declare an integer variable **n** in the Prime class.
5. Define a **parameterized constructor** in the **Prime** class that takes an integer parameter limit and assigns it to the **n** variable.
6. Define a method named **isprime(int n)** in the **Prime** class that checks whether a given number n is prime or not.
   - Iterate from 2 to n/2.
   - If n is divisible by any number in this range, **return 0** (indicating not prime).
   - Otherwise, **return 1** (indicating prime).
7. Override the **run**() method in the Prime class.
   - Inside the **run**() method, iterate from 1 to n.
   - For each iteration, check if the number is prime using the **isprime**() method.
   - If it's prime, print the prime number.
8. In the **main**() method:
   - Create an instance **m** of the **multiple5** class.
   - Prompt the user to enter a **limit** for generating prime numbers.
   - Create an instance **m1** of the **Prime** class with the input limit n.
   - **Start** both threads **m** and **m1**.

Source Code:

```
import java.util.*;
class multiple5 extends Thread
{
    public void run()
    {
        for(int i=1;i<=10;i++)
        {
            System.out.println(i+" x 5 = "+(i*5));
        }
    }
}
class Prime extends Thread
{
    int n;
```

```
    Prime(int limit)
    {
        n=limit;
    }
    int isprime(int n)
    {
        for(int i=2;i<=n/2;i++)
        {
            if(n%i==0)
            return 0;
        }
        return 1;
    }
    public void run()
    {
        for(int i=1;i<=n;i++)
        {
            if(isprime(i)==1)
                System.out.println("Prime: "+i);
        }
    }
}
public class Threadclass {
    public static void main(String[] args) {
        Scanner sc=new Scanner(System.in);
        multiple5 m =new multiple5();
        System.out.print("Enter limit for generating prime: ");
        int n=sc.nextInt();
        Prime m1 =new Prime(n);
        m.start();
        m1.start();
    }
}
```

Output:

```
Prime: 1
1 x 5 = 5
2 x 5 = 10
3 x 5 = 15
Prime: 2
4 x 5 = 20
Prime: 3
5 x 5 = 25
Prime: 5
6 x 5 = 30
Prime: 7
7 x 5 = 35
Prime: 11
8 x 5 = 40
Prime: 13
9 x 5 = 45
10 x 5 = 50
```

Result: The program has executed successfully and required output is obtained.

Date: 04/03/26

Program 18: **Runnable Interface**

Aim: Define 2 classes; one for generating Fibonacci numbers and other for displaying even numbers in a given range. Implement using threads. (Runnable Interface)

Algorithm:

1. Define a class named **fibonacci** that **implements** the **Runnable interface**.
2. Declare an integer variable **n** to store the limit.
3. Define a **parameterized constructor** in the **fibonacci** class that takes an integer parameter limit and assigns it to the **n** variable.
4. Implement the **run**() method in the fibonacci class.
   - Initialize variables **a** and **b** to **0** and **1** respectively.
   - Iterate from 1 to n.
   - **Print** the value of **a**.
   - Calculate the next Fibonacci number c by adding a and b.
   - Update a and b for the next iteration.
5. Define another class named **Even** that **implements** the **Runnable interface**.
6. Declare integer variables **start** and **end** in the Even class.
7. Define a **parameterized constructor** in the Even class that takes two integers **a** and **b** and assigns them to **start** and **end** respectively.
8. Implement the **run**() method in the Even class.
   - Iterate from start to end.
   - Check if the current number is even (i % 2 == 0).
   - If it's even, print the even number.
9. In the main():
10. Prompt the user to enter the number of Fibonacci numbers (n).
11. Prompt the user to enter the start and end range for even numbers.
12. Create two **threads f** and **e** with **instances** of **fibonacci** and **Even** classes respectively, passing necessary parameters.
13. **Start** both threads **f** and **e**.

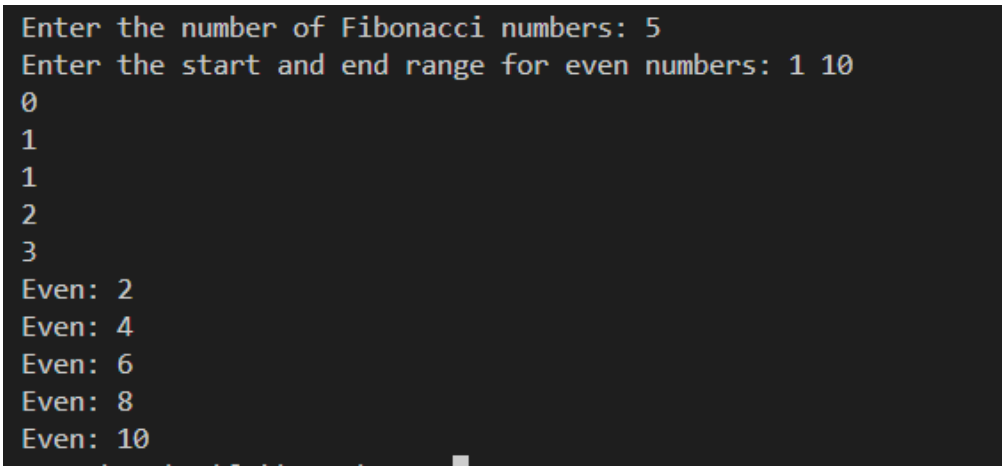
Source Code:

```
import java.util.*;
class fibonacci implements Runnable
{
    int n;
    fibonacci(int limit)
    {
        n=limit;
    }
    public void run()
    {
        int a = 0, b = 1, c;
        for (int i = 1; i <= n; i++)
```

```
            {
                System.out.println(a);
                c = a + b;
                a = b;
                b = c;
            }
        }
    }

    class Even implements Runnable
    {
        int start,end;
        Even(int a,int b)
        {
            start=a;
            end=b;
        }
        public void run()
        {
            for (int i = start; i <= end; i++)
            {
                if (i % 2 == 0)
                {
                    System.out.println("Even: "+i);
                }
            }
        }
    }
    public class Runnable_interface
    {
        public static void main(String[] args)
        {
            Scanner sc=new Scanner(System.in);
            System.out.print("Enter the number of Fibonacci numbers: ");
            int n = sc.nextInt();
            System.out.print("Enter the start and end range for even
                              numbers: ");
            int start = sc.nextInt();
            int end = sc.nextInt();
            Thread f=new Thread(new fibonacci(n));
            Thread e=new Thread(new Even(start,end));
            f.start();
```

```
            e.start();
        }
    }
```

Output:



Result: The program has executed successfully and required output is obtained.

Date: 04/03/26

Program 19: **Generic Stack**

Aim: Program to create a generic stack and do the Push and Pop operations.

Algorithm:

1. Define a generic class called stack with the following attributes:
   - **A**: An **ArrayList** to store elements of generic type T.
   - **top**: An integer representing the **index** of the **top** element in the stack. Initialized to -1.
   - **size**: An integer representing the **maximum size** of the stack.
2. Define a constructor that initializes the size of the stack and creates an ArrayList of the specified size.
3. Implement the **push** method:
   - Check if the stack is **full** (top + 1 == size).
   - If not, **increment** top **by** 1 and add the element to the ArrayList at the top index.
4. Implement the **top** method:
   - Check if the stack is **empty** (top == -1).
   - If not, **return** the **element at the top** index of the ArrayList.
5. Implement the **pop** method:
   - Check if the stack is empty (top == -1).
   - If not, **decrement top by 1.**
6. Implement the empty method:
   - Return true if the stack is empty (top == -1), otherwise return false.
7. Implement the **toString** method to represent the stack as a string:
   - **Initialize** an **empty string.**
   - Iterate over the elements in the ArrayList from index 0 to top - 1.
   - Append each element to the string followed by '->'.
   - Append the last element (at index top) to the string.
   - Return the constructed string.
8. Define the **main** method:
   - Prompt the user to enter the **maximum size** of the stack and read the input.
   - Create an **instance** of the **stack** class with the specified maximum size.
   - Use a loop to push elements into the stack:
   - Print the stack after pushing all elements.
   - Pop an element from the stack.
   - Print the stack after popping.

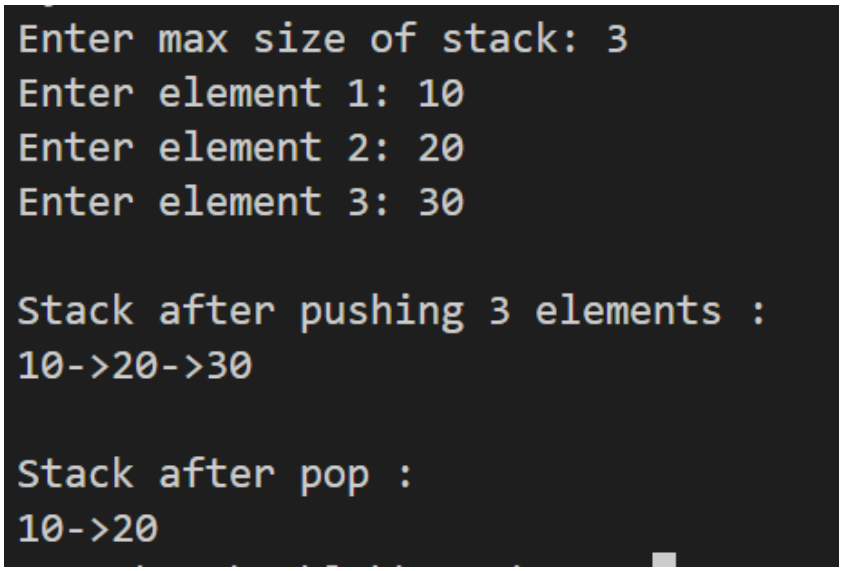
Source Code:

```
import java.util.*;
class stack<T>
{
    ArrayList<T> A;
    int top = -1;
    int size;
```

```
stack(int size)
{
    this.size = size;
    this.A = new ArrayList<T>(size);
}
void push(T X)
{
    if (top + 1 == size)
    {
        System.out.println("Stack Overflow");
    }
    else
    {
        top = top + 1;
        if (A.size() > top)
            A.set(top, X);
        else
            A.add(X);
    }
}
T top()
{
    if (top == -1)
    {
        System.out.println("Stack Underflow");
        return null;
    }
    else
        return A.get(top);
}
void pop()
{
    if (top == -1)
    {
        System.out.println("Stack Underflow");
    }
    else
        top--;
}
boolean empty() { return top == -1; }

public String toString()
```

```
        {

            String Ans = "";
            for (int i = 0; i < top; i++)
            {
                Ans += String.valueOf(A.get(i)) + "->";
            }
            Ans += String.valueOf(A.get(top));
            return Ans;
        }
    }
    public class GenericStack {
        public static void main(String[] args)
        {
            Scanner sc=new Scanner(System.in);
            System.out.print("Enter max size of stack: ");
            int n=sc.nextInt();
            stack<Integer> s1 = new stack<>(n);
            int v;
            for(int i=0; i<n; i++)
            {
                System.out.print("Enter element "+(i+1)+": ");
                v=sc.nextInt();
                s1.push(v);
            }

            System.out.println("\nStack after pushing "+n+" elements :\n" +
                               s1);
            s1.pop();
            System.out.println("\nStack after pop :\n" + s1);
        }
    }
```

Output:



Result: The program has executed successfully and required output is obtained.

Date: 11/03/26

Program 20: **ArrayList**

Aim: Program to maintain a list of Strings using ArrayList from collection framework, perform built-in operations.

Algorithm:

1. Create an empty **Arrayl.ist** of Strings called **list**.
2. Create variables **el** (to store user input for elements) and **ch** (to store user input for choices).
3. Start a do-while loop with the condition **ch** not equal to 0.
4. Inside the loop, display a menu of options and prompt the user to enter their choice.
5. On the basis user's choice:
    - Case 1: Prompt the user to enter an clement and add it to the **ArrayList** using **list_add(e1).**
    - Case 2: Print the size of the **Arraylist** using **list.size()**.
    - Case 3: Prompt the user to enter an **index** and print the element at that index using **list. get(index)**.
    - Case 4: Prompt the user to enter an **element** and **find its index** using **list.indexOf(el)**.
    - Case 5: Prompt the user to enter an **clement** and check **if it exists** in the ArrayList using **list.contains(el).**
    - Case 6. Prompt the user to enter an **element** and **remove** it from the ArrayList using **list.remove(el)**.
    - Case 7: Prompt the user to enter an **index** and **remove** the element at that index using **list.remove(index).**
    - Case 8: **Print** the **entire ArrayList** using **'list'**.
    - Case 9: **Clear** the **ArrayList** using **list. clear( )**
    - Case 0: Exit the loop.

Source Code:

```
import java.util.*;
public class Arraylist {
    public static void main(String[] args) {
        ArrayList <String> list = new ArrayList<String>();
        Scanner sc=new Scanner(System.in);
        String el;
        int ch;
        do
        {
            System.out.print("\n----------------\n1: add\n2: size\n3:
            search by index\n4: find index\n5: contains\n6: remove\n7:
            remove by index\n8: display\n9: clear search\n0: Exit\n------
            ----------\nEnter your choice: ");
            ch=sc.nextInt();
            switch(ch)
            {
```

```
                case 1:
                    System.out.print("Enter element to insert: ");
                    el=sc.next();
                    list.add(el);
                    break;
                case 2:
                    System.out.println("Size of the arraylist:
                                        "+list.size());
                    break;
                case 3:
                    System.out.print("Enter index of element to search:
                                       ");
                    int index=sc.nextInt();
                    System.out.println("Element at index "+index+" is
                                        "+list.get(index));
                    break;
                case 4:
                    System.out.print("Enter an element to find index:
                                      ");
                    el=sc.next();
                    index=list.indexOf(el);
                    System.out.println("Index of "+el+" is "+index);
                    break;
                case 5:
                    System.out.print("Enter a element");
                    el=sc.next();
                    boolean contains=list.contains(el);
                    System.out.println(el+" is in the list :"+contains);
                    break;
                case 6:
                    System.out.print("Enter the element to be removed:
                                      ");
                    el=sc.next();
                    boolean removed=list.remove(el);
                    System.out.println("After removing "+el+ " ArrayList
                                        : "+list);
                    break;
                case 7:
                    System.out.print("Enter the index to remove the
                                      element: ");
                    index=sc.nextInt();
                    list.remove(index);
```

```
                    System.out.println("After removing the element,
                                        arraylist:"+list);
                    break;
                case 8:
                    System.out.println("Arraylist: "+list);
                    break;
                case 9:
                    list.clear();
                    break;
                case 0:
                    break;
            }
        }while(ch!=0);
    }
}
```

Output:

```
----------------
1: add
2: size
3: search by index
4: find index
5: contains
6: remove
7: remove by index
8: display
9: clear search
0: Exit
----------------
Enter your choice: 1
Enter element to insert: 1
```

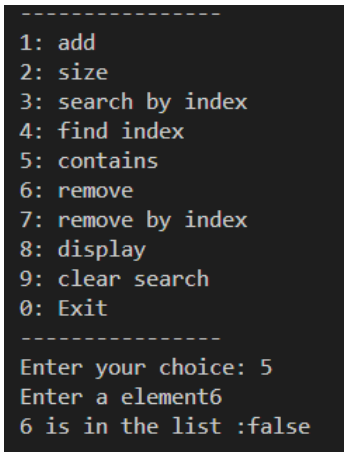
```
----------------
1: add
2: size
3: search by index
4: find index
5: contains
6: remove
7: remove by index
8: display
9: clear search
0: Exit
----------------
Enter your choice: 1
Enter element to insert: 2
```

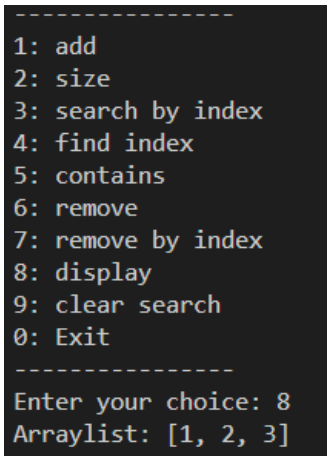
```
----------------
1: add
2: size
3: search by index
4: find index
5: contains
6: remove
7: remove by index
8: display
9: clear search
0: Exit
----------------
Enter your choice: 1
Enter element to insert: 3
```

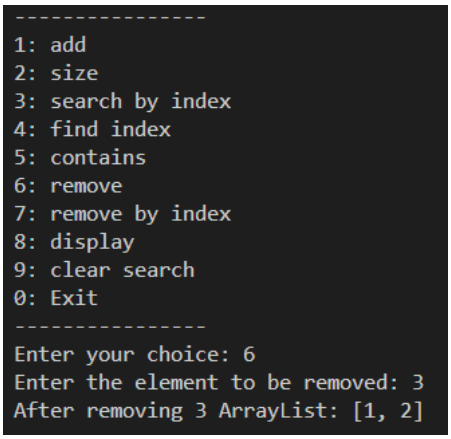
```
----------------
1: add
2: size
3: search by index
4: find index
5: contains
6: remove
7: remove by index
8: display
9: clear search
0: Exit
----------------
Enter your choice: 2
Size of the arraylist: 3
```

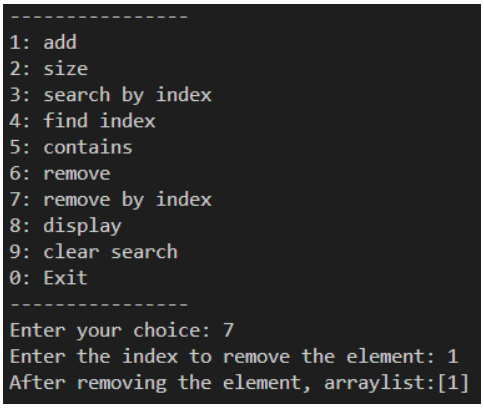
```
----------------
1: add
2: size
3: search by index
4: find index
5: contains
6: remove
7: remove by index
8: display
9: clear search
0: Exit
----------------
Enter your choice: 3
Enter index of element to search: 2
Element at index 2 is 3
```

```
----------------
1: add
2: size
3: search by index
4: find index
5: contains
6: remove
7: remove by index
8: display
9: clear search
0: Exit
----------------
Enter your choice: 4
Enter an element to find index: 2
Index of 2 is 1
```









Result: The program has executed successfully and required output is obtained.

Date: 11/03/26

Program 21: **Queue**

Aim: Program to demonstrate the creation of queue object using the PriorityQueue class.

Algorithm:

1. Create a new **PriorityQueue** object called q to store strings.
2. Declare variables el and ch of type String and int respectively.
3. Based on the users choice
   - Case I: **Add** :
     - Prompt the user to enter the clement to insert and store it in the variable cl.
     - Call the add method on the q object, passing el as the argument.
   - Case 2: **Remove**
     - Prompt the user to enter the clement to remove and store it in the variable el.
     - Call the remove method on the q object, passing el as the argument .
   - Case 3: **Display**
     - Print the contents of the q priority queue.
   - Case 4: **Head**
     - Print the head of the q priority queue using the peck method.
   - Case 5: **Poll**
     - Remove and return the head of the q priority queue using the poll method.
   - Case 0: **Wrong choice**

Source Code:

```
import java.util.*;
public class Queue
{
    public static void main(String[] args)
    {
        PriorityQueue<String>q=new PriorityQueue<String>();
        Scanner sc=new Scanner(System.in);
        String el;
        int ch;
        do
        {
            System.out.print("\n----------------\n1:add\n2:remove
                             \n3:display\n4:head\0:wrong choice\n-------
                             ---------\n Enter your choice: ");
            ch=sc.nextInt();
            switch(ch)
            {
                case 1:
                    System.out.print("Enter element to insert:\n");
                    el=sc.next();
```

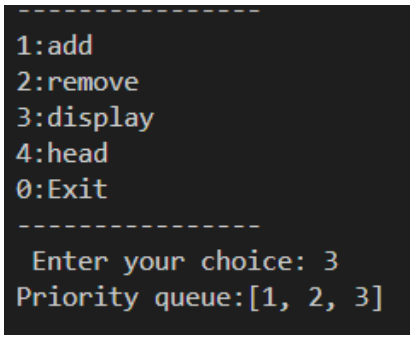
```
                    q.add(el);
                    break;
                case 2:
                    q. remove();
                    break;
                case 3:
                    System.out.println("Priority queue:"+q);
                    break;
                case 4:
                    System.out.println("Head of the queue:"+q.peek());
                    break;
                case 9:break;
            }
        }while(ch!=0);
    }
}
```

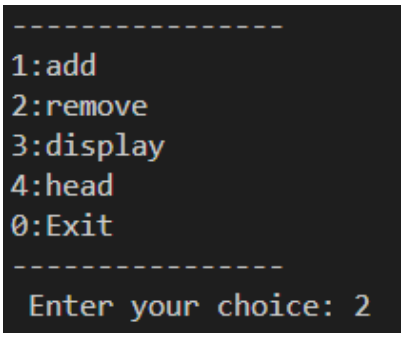
Output:

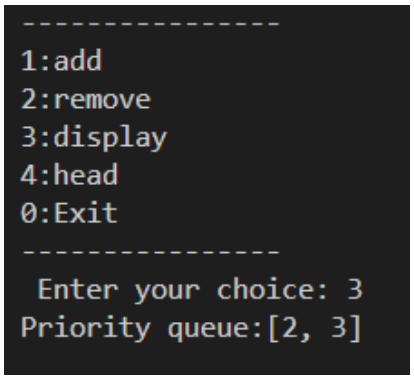
```
---------------
1:add
2:remove
3:display
4:head
0:Exit
---------------
 Enter your choice: 1
Enter element to insert: 1
```

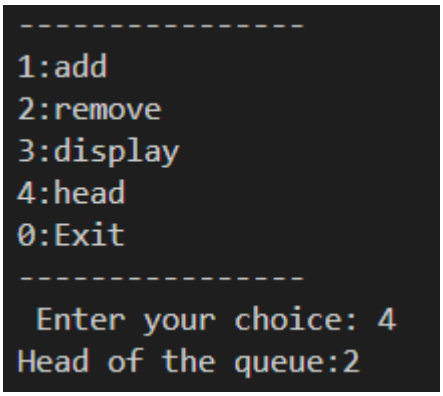
```
----------------
1:add
2:remove
3:display
4:head
0:Exit
----------------
 Enter your choice: 1
Enter element to insert: 2
```

```
----------------
1:add
2:remove
3:display
4:head
0:Exit
----------------
 Enter your choice: 1
Enter element to insert: 3
```









Result: The program has executed successfully and required output is obtained.

Date: 11/03/26

Program 22: **Set Using Linked Hashset**

Aim: Program to demonstrate the creation of Set object using the LinkedHashset class.

Algorithm:

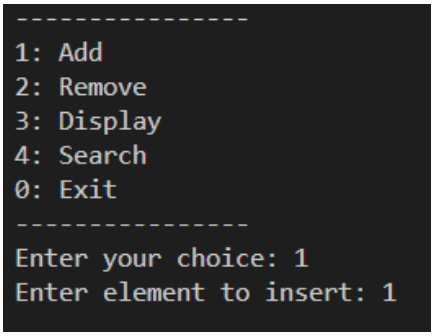
1. Create a new **LinkedHashSet** object called set to store strings.
2. Declare variables **el** and **ch** of type String and int respectively.
3. Based on the users choice:
    - Case I: **Add**
        - Prompt the user to enter the element to insert and store it in the variable el.
        - Call the add method on the set object, passing el as the argument.
    - Case 2: **Remove**
        - Prompt the user to enter the element to remove and store it in the variable el.
        - Call the remove method on the set object, passing el as the argument.
    - Case 3: **Display**
        - Print the contents of the set linked hash set.
    - Case 4: **Search**
        - Prompt the user to enter the element to search and store it in the variable el.
        - Call the contains method on the set object, passing el as the argument, and store the result in a boolean variable contains.
        - Print whether the set contains the element or not.
    - Case 0: **Exit**
        - Break out of the switch statement.

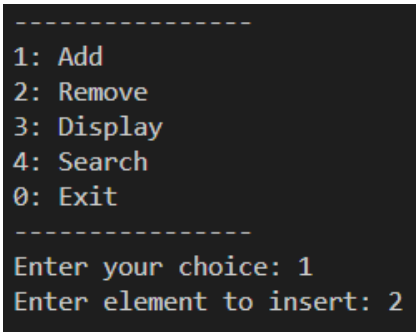
Source Code:

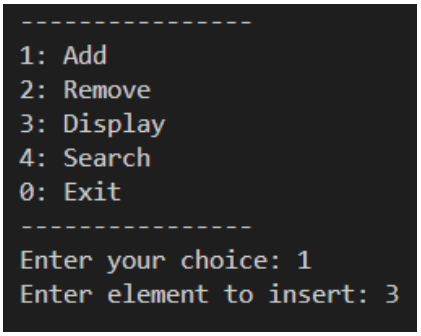
```
import java.util.*;
public class Hashset
{
    public static void main(String[] args)
    {
        Set <String> set =new LinkedHashSet<String>();
        Scanner sc=new Scanner(System.in);
        String el;
        int ch;
        do
        {
            System.out.print("\n----------------\n1: Add\n2: Remove\n3:
            Display\n4: Search\n0: Exit\n----------------\nEnter your
            choice: ");
            ch=sc.nextInt();
            switch(ch)
            {
                case 1:
```

```
                    System.out.print("Enter element to insert: ");
                    el=sc.next();
                    set.add(el);
                    break;
                case 2:
                    System.out.print("Enter element to remove: ");
                    el=sc.next();
                    set.remove(el);
                    break;
                case 3:
                    System.out.println("Linked Hashset: "+set);
                    break;
                case 4:
                    System.out.print("Enter element to search: ");
                    el=sc.next();
                    boolean contains=set.contains(el);
                    System.out.println("Set contains "+el+" : "+contains);
                    break;
                case 0:
                    System.out.println("Exiting");
                    break;
            }
        }while(ch!=0);
    }
}
```

Output:







```
----------------
1: Add
2: Remove
3: Display
4: Search
0: Exit
----------------
Enter your choice: 3
Linked Hashset: [1, 2, 3]
```

```
----------------
1: Add
2: Remove
3: Display
4: Search
0: Exit
----------------
Enter your choice: 2
Enter element to remove: 2
```

```
----------------
1: Add
2: Remove
3: Display
4: Search
0: Exit
----------------
Enter your choice: 4
Enter element to search: 2
Set contains 2 : false
```

Result: The program has executed successfully and required output is obtained.

Date: 18/03/26

Program 23: **Simple Calculator -AWT**

Aim: Implement a simple calculator using AWT components

Algorithm:

1. Import necessary packages: **java.awt.*** and **java.awt.event.***.
2. Declare the **SimpleCalculator** class, which **extends Frame** and **implements ActionListener**.
3. Define class variables:
   - **textField**: TextField for displaying input and result.
   - **numberButtons**: Array of Buttons for numbers 0-9.
   - **operationButtons**: Array of Buttons for operations (+, -, *, /).
   - **equalsButton**: Button for performing the calculation.
   - **clearButton**: Button for clearing the text field.
   - **num1**, **num2**: Doubles to store operands.
   - **result**: Double to store the result of the operation.
   - **operation**: Character to store the current operation.
4. Implement the **constructor** SimpleCalculator():
   - Set the **title** and **size** of the **frame**.
   - Set layout to BorderLayout.
   - Create and configure the textField, add it to the NORTH of the frame.
   - Create **buttonPanel** with **GridLayout(4, 4).**
   - Create **numberButtons** for digits 0-9, add ActionListener, and add them to **buttonPanel**.
   - Create **operationButtons** for operations (+, -, *, /), add ActionListener, and add them to **buttonPanel**.
   - Create **equalsButton** with ActionListener and add it to **buttonPanel**.
   - Create **clearButton** with ActionListener and add it to **buttonPanel**.
   - Add **buttonPanel** to the CENTER of the frame.
   - Add window closing listener to exit the application.
5. Implement **actionPerformed(ActionEvent ae)** method:
6. Get the action command from the event.
7. If the command is a digit (0-9), append it to the text field.
8. If the command is 'C', clear the text field.
9. If the command is '=', perform the calculation based on the stored operation and display the result in the text field.
10. If the command is an operation (+, -, *, /), store the current value in num1, store the operation, and clear the text field.
11. In the main method:
    - Create an instance of SimpleCalculator.
    - Make the calculator visible.

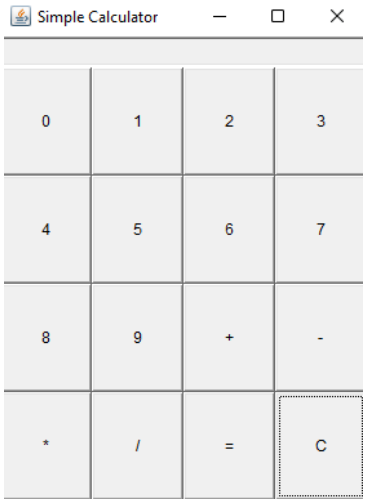
Source Code:

```
import java.awt.*;
import java.awt.event.*;

public class SimpleCalculator extends Frame implements ActionListener
{
    private TextField textField;
    private Button[] numberButtons;
    private Button[] operationButtons;
    private Button equalsButton;
    private Button clearButton;
    private double num1, num2, result;
    private char operation;

    public SimpleCalculator()
    {
        setTitle("Simple Calculator");
        setSize(300, 400);
        setLayout(new BorderLayout());

        textField = new TextField();
        textField.setEditable(false);
        add(textField, BorderLayout.NORTH);

        Panel buttonPanel = new Panel();
        buttonPanel.setLayout(new GridLayout(4, 4));

        numberButtons = new Button[10];
        for (int i = 0; i < 10; i++)
        {
            numberButtons[i] = new Button(String.valueOf(i));
            numberButtons[i].addActionListener(this);
            buttonPanel.add(numberButtons[i]);
        }

        operationButtons = new Button[4];
        operationButtons[0] = new Button("+");
        operationButtons[1] = new Button("-");
        operationButtons[2] = new Button("*");
        operationButtons[3] = new Button("/");
        for (int i = 0; i < 4; i++)
```

```
        {
            operationButtons[i].addActionListener(this);
            buttonPanel.add(operationButtons[i]);
        }

        equalsButton = new Button("=");
        equalsButton.addActionListener(this);
        buttonPanel.add(equalsButton);

        clearButton = new Button("C");
        clearButton.addActionListener(this);
        buttonPanel.add(clearButton);

        add(buttonPanel, BorderLayout.CENTER);

        addWindowListener(new WindowAdapter()
        {
            public void windowClosing(WindowEvent windowEvent)
            {
                System.exit(0);
            }
        });
    }

    public void actionPerformed(ActionEvent ae)
    {
        String command = ae.getActionCommand();
        if (command.charAt(0) >= '0' && command.charAt(0) <= '9')
            textField.setText(textField.getText() + command);
        else if (command.charAt(0) == 'C')
            textField.setText("");
         else if (command.charAt(0) == '=')
        {
            num2 = Double.parseDouble(textField.getText());
            switch (operation)
            {
                case '+':
                    result = num1 + num2;
                    break;
                case '-':
                    result = num1 - num2;
                    break;
```

```
                case '*':
                    result = num1 * num2;
                    break;
                case '/':
                    if (num2 != 0)
                        result = num1 / num2;
                    else
                        textField.setText("Cannot divide by zero");
                    break;
            }
            textField.setText(String.valueOf(result));
        }
        else
        {
            num1 = Double.parseDouble(textField.getText());
            operation = command.charAt(0);
            textField.setText("");
        }
    }

    public static void main(String[] args)
    {
        SimpleCalculator calculator = new SimpleCalculator();
        calculator.setVisible(true);
    }
}
```

Output:



Result: The program has executed successfully and required output is obtained.

Date: 18/03/26

Program 24: **Mouse Events**

Aim: Develop a program to handle all mouse events and window events.

Algorithm:

1. Import the required libraries**: javax.swing.*** and **java.awt.event.*.**
2. Define a class named **MouseEventDemo** that extends **JFrame**.
3. Define the constructor:
   - Set the title of the window to "Mouse Events Demo".
   - Set the size of the window to 400x300 pixels.
   - Set the default close operation to exit on close.
4. Add a MouseListener to the frame using **addMouseListener(new MouseAdapter() { ... })**.
5. Override methods for the following mouse events:
   - **mouseClicked(MouseEvent e):** Print the coordinates of the mouse when clicked.
   - **mousePressed(MouseEvent e)**: Print the coordinates of the mouse when pressed.
   - **mouseReleased(MouseEvent e)**: Print the coordinates of the mouse when released.
   - **mouseEntered(MouseEvent e)**: Print the coordinates of the mouse when entered into the frame.
   - **mouseExited(MouseEvent e)**: Print the coordinates of the mouse when exited from the frame.
6. Add a **WindowListener** to the frame using **addWindowListener(new WindowAdapter() { ... })**.
7. Override **the windowClosing(WindowEvent e)** method to print a message when the window is closed.
8. In the **main** method:
   - Use SwingUtilities.invokeLater() to ensure that Swing components are created and accessed in the Event Dispatch Thread for thread safety.
   - Create an instance of MouseEventDemo.
   - Set the visibility of the frame to true.

Source Code:

```
import javax.swing.*;
import java.awt.event.*;

public class MouseEventDemo extends JFrame
{
    public MouseEventDemo()
    {
        setTitle("Mouse Events Demo");
        setSize(400, 300);
        setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);

        addMouseListener(new MouseAdapter()
```

```
        {
            public void mouseClicked(MouseEvent e)
            {
                System.out.println("Mouse clicked at (" + e.getX() + ",
                                   " + e.getY() + ")");
            }

            public void mousePressed(MouseEvent e)
            {
                System.out.println("Mouse pressed at (" + e.getX() + ",
                                   " + e.getY() + ")");
            }

            public void mouseReleased(MouseEvent e)
            {
                System.out.println("Mouse released at (" + e.getX() + ",
                                   " + e.getY() + ")");
            }

            public void mouseEntered(MouseEvent e)
            {
                System.out.println("Mouse entered at (" + e.getX() + ",
                                   " + e.getY() + ")");
            }

            public void mouseExited(MouseEvent e)
            {
                System.out.println("Mouse exited at (" + e.getX() + ", "
                                   + e.getY() + ")");
            }
        });

        addMouseMotionListener(new MouseMotionAdapter()
        {
            public void mouseDragged(MouseEvent e)
            {
                System.out.println("Mouse dragged to (" + e.getX() + ",
                                   " + e.getY() + ")");
            }

            public void mouseMoved(MouseEvent e)
            {
```

```
                System.out.println("Mouse moved to (" + e.getX() + ", "
                                   + e.getY() + ")");
            }
        });

        addWindowListener(new WindowAdapter()
        {
            public void windowClosing(WindowEvent e)
            {
                System.out.println("Window closed");
            }
        });
    }

    public static void main(String[] args)
    {
        SwingUtilities.invokeLater(() ->
        {
            MouseEventDemo demo = new MouseEventDemo();
            demo.setVisible(true);
        });
    }
}
```

Output:

```
Mouse entered at (209, 31)
Mouse moved to (209, 31)
Mouse moved to (209, 32)
Mouse pressed at (209, 32)
Mouse released at (209, 32)
Mouse clicked at (209, 32)
Mouse pressed at (209, 32)
Mouse released at (209, 32)
Mouse clicked at (209, 32)
Mouse moved to (209, 31)
Mouse exited at (209, 30)
Window closed
```

Result: The program has executed successfully and required output is obtained.

Date: 25/03/26

Program 25: **Subdirectory and Files**

Aim: Program to list the sub directories and files in a given directory and also search for a file name.

Algorithm:

1. Define class **DirectoryListing**.
2. Define static method **listFilesAndDirectories(directory: File):**
   - For each file in directory:
   - Print file name.
   - If file is a directory, call listFilesAndDirectories recursively.
3. Define static method **searchFile(directory: File, fileName: String):**
   - For each file in directory:
   - If file matches fileName, print its absolute path.
   - If file is a directory, call searchFile recursively.
   - If file is not found, print a message indicating so.
4. Define **main** method:
   - Read directory path from user input.
   - Create File object directory with directory path.
   - If directory exists and is a directory:
   - Print files and directories in specified directory.
   - Prompt user to enter file name to search.
   - Search for file in directory.
   - If directory does not exist or is not a directory, print message indicating invalid directory path.

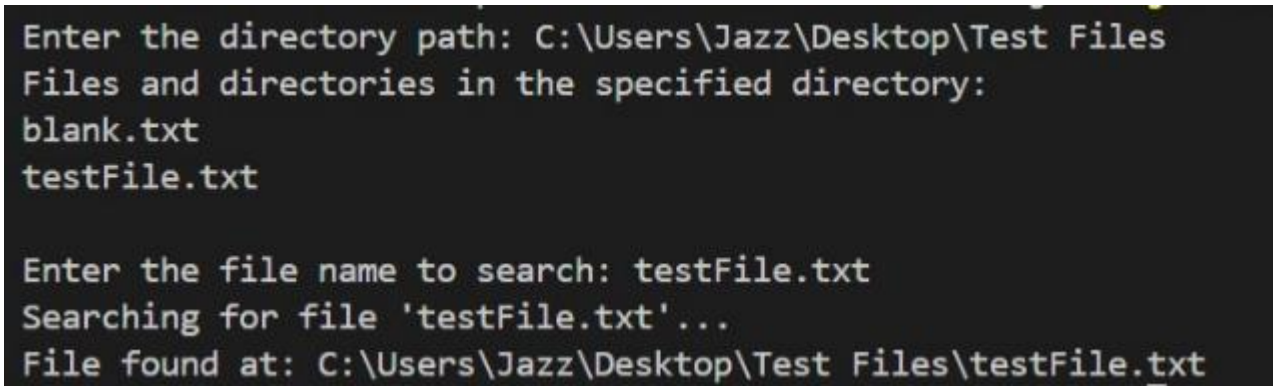
Source Code:

```
import java.io.File;
import java.util.Scanner;

public class DirectoryListing
{
    static void listFilesAndDirectories(File directory)
    {
        File[] fileList = directory.listFiles();
        if (fileList != null)
        {
            for (File file : fileList)
            {
                System.out.println(file.getName());
                if (file.isDirectory())
                    listFilesAndDirectories(file);
```

```
            }
        }
    }

    static void searchFile(File directory, String fileName)
    {
        File[] fileList = directory.listFiles();
        if (fileList != null)
        {
            for (File file : fileList)
            {
                if (file.isFile() && file.getName().equals(fileName))
                {
                    System.out.println("File found at: " +
                                        file.getAbsolutePath());
                    return;
                }
                else if (file.isDirectory())
                    searchFile(file, fileName);
            }
        }
        System.out.println("File '" + fileName + "' not found.");
    }

    public static void main(String[] args)
    {
        Scanner scanner = new Scanner(System.in);
        System.out.print("Enter the directory path: ");
        String directoryPath = scanner.nextLine();
        File directory = new File(directoryPath);
        if (directory.exists() && directory.isDirectory())
        {
            System.out.println("Files and directories in the specified
                                directory:");
            listFilesAndDirectories(directory);
            System.out.print("\nEnter the file name to search: ");
            String fileName = scanner.nextLine();
            System.out.println("Searching for file '" + fileName +
                               "'...");
            searchFile(directory, fileName);
        }
        else
```

```
            System.out.println("Invalid directory path.");
            scanner.close();
        }
    }
```

Output:



Result: The program has executed successfully and required output is obtained.

Date: 25/03/26

Program 26: **Files: Write and Read**

Aim: Write a program to write to a file, then read from the file and display the contents on the console.

Algorithm:

1. Prompt the user to enter the file name and store it in **fileName**.
2. Create a **FileOutputStream (fos) with fileName**.
3. Prompt the user to enter text to insert and store it in **text**.
4. Write text to fos.
5. Close fos.
6. Create a **FileInputStream (fis) with fileName.**
7. Read bytes from **fis** into a byte array (**b**).
8. Close fis.
9. Convert byte array (b) to a string (**fileContent**).
10. Print "Contents of " + fileName + ":".
11. Print fileContent.

Source Code:

```
import java.io.*;
import java.util.*;
public class WriteRead
{
    public static void main(String[] args)
    {
        Scanner sc = new Scanner(System.in);
        System.out.print("Enter the file name: ");
        String fileName = sc.nextLine();
        FileOutputStream fos = new FileOutputStream(fileName);
        System.out.println("Enter text to insert: ");
        String text = sc.nextLine();
        fos.write(text.getBytes());
        FileInputStream fis = new FileInputStream(fileName);
        byte[] b = new byte[fis.available()];
        fis.read(b);
        fis.close();
        String fileContent = new String(b);
        System.out.println("Contents of " + fileName + ":");
        System.out.println(fileContent);
        sc.close();
    }
}
```

Output:

```
Enter the file name: testfile
Enter text to insert:
Test Data
Contents of testfile:
Test Data
```

Result: The program has executed successfully and required output is obtained.

Date: 01/04/26

Program 27: **Copy File to Another**

Aim: Write a program to copy one file to another.

Algorithm:

1. Create **BufferedReader** (reader) for user input.
2. Prompt user for source file path, store in **sourceFile**.
3. Prompt user for destination file path, store in **destinationFile**.
4. Open **input stream (in)** for source file, **output stream (out)** for destination file.
5. Read from input stream into buffer, write to output stream until end of file.
6. Close input and output streams.
7. Close BufferedReader.

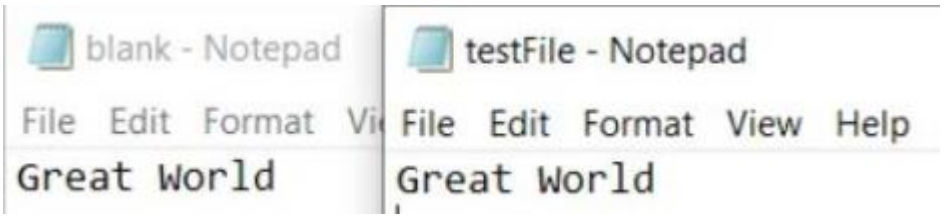
Source Code:

```
import java.io.*;
public class FileCopy
{
    public static void main(String[] args) throws IOException
    {
        BufferedReader reader = new BufferedReader(new InputStreamReader
                                                    (System.in));
        System.out.print("Enter the source file path: ");
        String sourceFile = reader.readLine();
        System.out.print("Enter the destination file path: ");
        String destinationFile = reader.readLine();
        try (InputStream in = new FileInputStream(sourceFile);
             OutputStream out = new FileOutputStream(destinationFile))
        {
            byte[] buffer = new byte[1024];
            int bytesRead;
            while ((bytesRead = in.read(buffer)) != -1)
            {
                out.write(buffer, 0, bytesRead);
            }
            System.out.println("File copied successfully.");
        }
        catch (IOException e)
        {
            System.out.println("An error occurred during file copy: " +
                               e.getMessage());
        }
        finally
```

```
            {
                reader.close();
            }
        }
    }
```

Output:

```
Enter the source file path: \Users\Jazz\Desktop\Test Files\testFile.txt
Enter the destination file path: \Users\Jazz\Desktop\Test Files\blank.txt
File copied successfully.
```



Result: The program has executed successfully and required output is obtained.

Date: 01/04/26

Program 28: **Files: Even and Odd**

Aim: Write a program that reads from a file having integers. Copy even numbers and odd numbers to separate files.

Algorithm:

1. Create a **BufferedReader** (reader) to read user input.
2. Prompt the user to enter the input file path and store it in **inputFile**.
3. Define file paths for **evenFile** and **oddFile**.
4. Create a **BufferedReader (fileReader)** to read from the input file.
5. Create a **BufferedWriter (evenWriter)** to write even numbers to the even file.
6. Create a **BufferedWriter (oddWriter)** to write odd numbers to the odd file.
7. Read each line from the input file, parse it to an integer, and determine if it's even or odd.
8. Write even numbers to the even file and odd numbers to the odd file.
9. Print a success message indicating the separation is complete.
10. Close the BufferedReader, BufferedWriter for even numbers, and BufferedWriter for odd numbers.
11. Close the BufferedReader for user input.

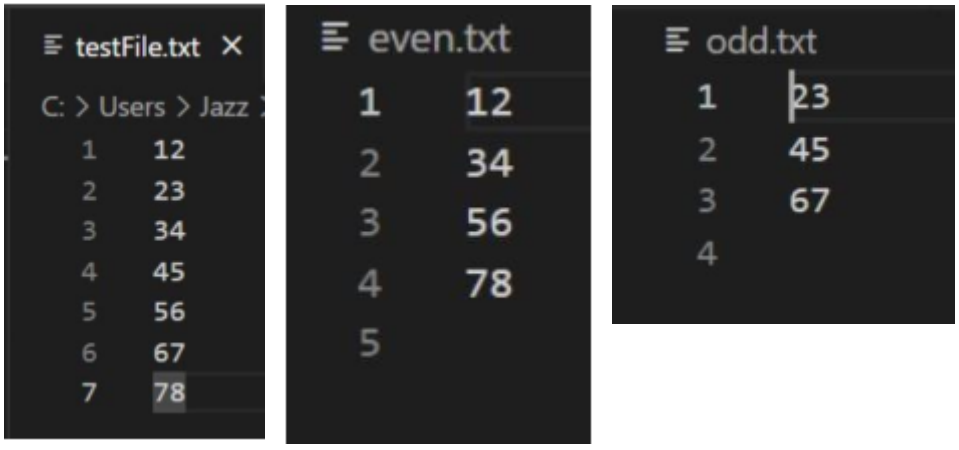
Source Code:

```
import java.io.*;
public class NumberSeparation
{
    public static void main(String[] args)
    {
        BufferedReader reader = new BufferedReader(new InputStreamReader
                                                    (System.in));
        System.out.print("Enter the input file path: ");
        String inputFile = reader.readLine();
        String evenFile = "even.txt";
        String oddFile = "odd.txt";
        BufferedReader fileReader = new BufferedReader(new FileReader
                                                        (inputFile));
        BufferedWriter evenWriter = new BufferedWriter(new FileWriter
                                                        (evenFile));
        BufferedWriter oddWriter = new BufferedWriter(new FileWriter
                                                        (oddFile));
        String line;
        while ((line = fileReader.readLine()) != null)
        {
            int number = Integer.parseInt(line);
```

```
                if (number % 2 == 0)
                {
                    evenWriter.write(line);
                    evenWriter.newLine();
                }
                else
                {
                    oddWriter.write(line);
                    oddWriter.newLine();
                }
            }
            System.out.println("Even and odd numbers separated successfully.
                               ");
            reader.close();
            fileReader.close();
            evenWriter.close();
            oddWriter.close();
        }
    }
```

Output:

```
Enter the input file path: C:\Users\Jazz\Desktop\Test Files\testFile.txt
Even and odd numbers separated successfully.
```



Result: The program has executed successfully and required output is obtained.